SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging Test

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 3 – Debugging Test
Weightage:		**Total Marks:**	25 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)

Student Name:	Sandra Sudevan	**Reg. No.:**	192424422
Section / Batch:	2024	**Date of Submission:**	4/08/26
Faculty In-charge:	**Dr. B.SHYAMALA GOWRI**	**Project Mode:**	Individual Submission

Code of Conduct

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sandra Sudevan

Error Identification(5)	Code Correction(8)	SQL Query Accuracy(5)	Explanation and Justification(4)	Program Execution and Output(3)	Total (25 Marks)

Q2. Debug the CRUD Program import sqlite3
conn = sqlite3.connect("college.db")cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchone():
    print(row)
cursor.execute("UPDATE Student SET marks=95")
cursor.execute("DELETE FROM Student")
conn.close()
Task:

1. Identify all logical and syntax errors.
2. Explain why every record is modified or deleted.
3. Rewrite the corrected program.

AIM

To debug and execute a Python SQLite CRUD program using the Student table.

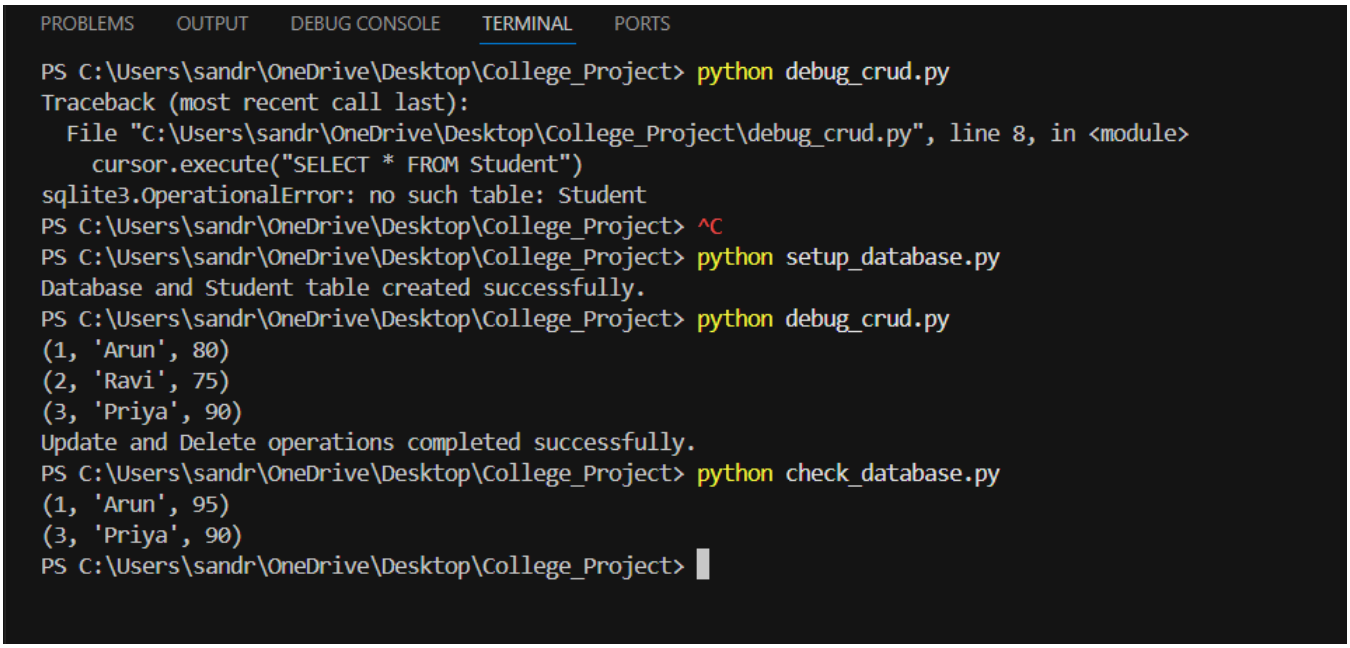
PROCEDURE

1. Create college.db and the Student table.
2. Insert sample student records.
3. Write the corrected CRUD program in debug_crud.py.
4. Use SELECT, UPDATE ... WHERE, and DELETE ... WHERE.
5. python debug_crud.py
6. Use commit() to save changes.
7. Verify the final output and close the database.

Corrected Code

```
import sqlite3

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

cursor.execute("SELECT * FROM Student")

for row in cursor.fetchall():

    print(row)

cursor.execute("UPDATE Student SET marks=95 WHERE StudentID=1")

cursor.execute("DELETE FROM Student WHERE StudentID=2")

conn.commit()

conn.close()
```

print("Update and Delete completed successfully.")



Errors Corrected

- Fixed missing line break between connect() and cursor().
- Changed fetchone() to fetchall().
- Added WHERE to UPDATE so all records are not modified.
- Added WHERE to DELETE so all records are not deleted.
- Added commit() to save changes.

Result

Thus, the errors in the SQLite CRUD program were identified and corrected successfully. The Student records were displayed, one student's marks were updated, and one student record was deleted successfully.

Q7. Debug the Parameterized Query id=2
cursor.execute("SELECT * FROM Student WHERE id="+id)
print(cursor.fetchall())
Task

1. Identify all errors.
2. Explain why the code is unsafe.
3. Rewrite the query using parameterized SQL.

AIM

To execute a parameterized SQL query in Python SQLite and retrieve student records safely.

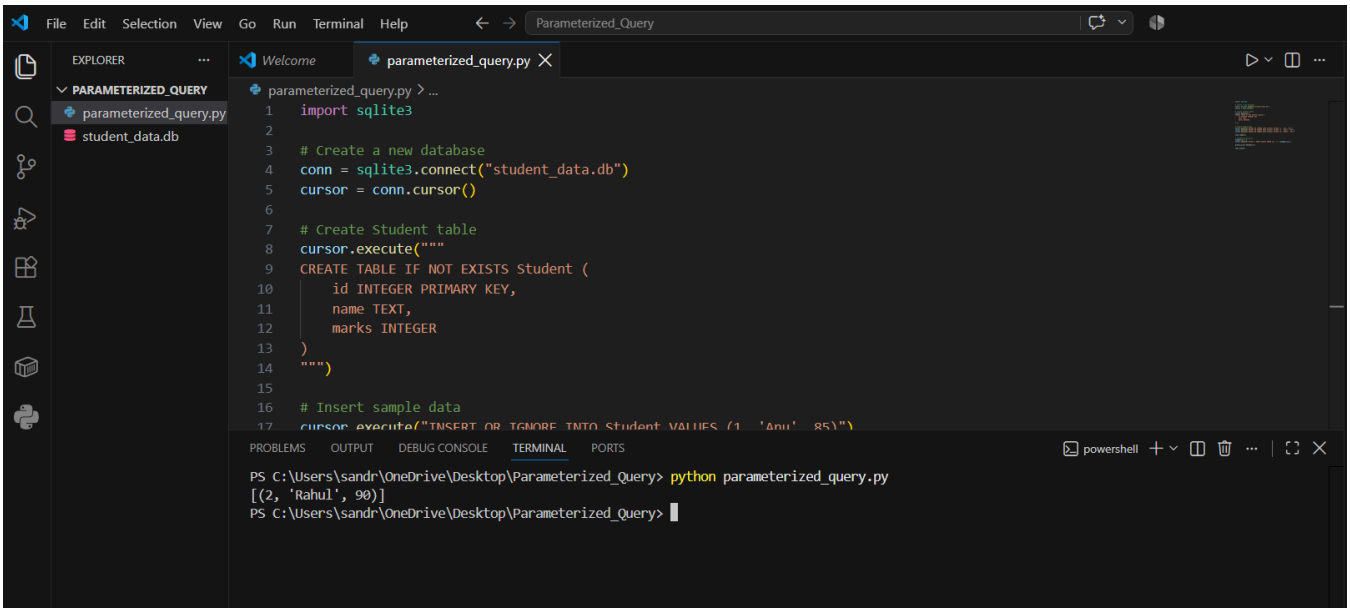
PROCEDURE

1. Create a folder named Parameterized_Query in VS Code.
2. Create parameterized_query.py.
3. Create a SQLite database and Student table using Python.
4. Insert sample student records.
5. Use ? as a parameterized query placeholder.
6. Run:
   python parameterized_query.py
7. Display the matching student record and verify the output.

CORRECTED CODE

```
import sqlite3

conn = sqlite3.connect("student_data.db")

cursor = conn.cursor()

cursor.execute("""

CREATE TABLE IF NOT EXISTS Student (

   id INTEGER PRIMARY KEY,

   name TEXT,

   marks INTEGER

)

""")

cursor.execute("INSERT OR IGNORE INTO Student VALUES (1, 'Anu', 85)")

cursor.execute("INSERT OR IGNORE INTO Student VALUES (2, 'Rahul', 90)")
```

```
cursor.execute("INSERT OR IGNORE INTO Student VALUES (3, 'Meena', 88)")

conn.commit()

student_id = 2

cursor.execute("SELECT * FROM Student WHERE id = ?", (student_id,))

print(cursor.fetchall())

conn.close()
```



Errors

- Directly concatenating id with the SQL query is unsafe.
- It can cause a TypeError if id is an integer.
- It may allow SQL Injection.
- The query should use ? as a parameter placeholder.

Result

Thus, the errors were identified and corrected successfully, and the parameterized SQL query executed safely to retrieve the required student record.